

**Міністерство освіти і науки України  
Національний технічний університет України  
«Київський політехнічний інститут імені Ігоря Сікорського»  
Факультет інформатики та обчислювальної техніки  
Кафедра обчислювальної техніки**

**Лабораторна робота №5  
з дисципліни  
«Основи розробки програмного забезпечення мовою  
програмування Python»  
на тему  
“Робота з текстовими файлами: читання та запис”**

Виконав:  
студент групи ІМ-42  
Жила Іван Дмитрович  
номер варіанту: 8

Перевірив:  
Фещенко К. Ю

Київ 2026

## **Мета роботи**

Ознайомитися з основами роботи з файлами у Python: відкрити файл для читання та запису, зчитувати та модифікувати його вміст, здійснювати обробку рядків і запис результатів у новий файл.

## **Короткі теоретичні відомості:**

### **Основи відкриття та закриття файлів**

- Файл у Python — це зовнішній ресурс для збереження інформації у постійне сховище (текстове або бінарне).
- Для відкриття файлів використовується функція `open('назва_файлу', 'режим', encoding='utf-8')`.
- Після завершення роботи файл необхідно закрити за допомогою методу `close()`, щоб звільнити ресурси.
- Для безпечної роботи з файлами та їх автоматичного закриття використовується контекстний менеджер `with` (наприклад, `with open(...) as f:`).
- Перед читанням варто перевіряти наявність файлу за допомогою методу `os.path.exists()`.

### **Режими роботи з файлами**

- «r» — читання (файл повинен існувати)
- 'w' — запис із перезаписом (якщо файл існував, його попередній вміст буде стерто).
- «a» — дозапис (нові дані безпечно додаються в кінець файлу).
- 'r+' — читання та запис одночасно.

### **Зчитування даних**

- `read()` — зчитує абсолютно весь вміст файлу і повертає його як один великий рядок.
- `readline()` — зчитує лише один рядок за раз.
- `readlines()` — зчитує всі рядки файлу і повертає їх у вигляді списку.

### **Запис даних**

- `write()` — записує один переданий рядок тексту у файл.
- `writelines()` — записує список рядків у файл.

- При використанні `writelines()` перенесення на новий рядок (`\n`) потрібно вказувати та додавати вручну.

### Обробка текстових даних

- При зчитуванні рядків можна одразу виконувати їх обробку вбудованими методами Python.
- Методи `upper()` та `lower()` використовуються для зміни регістру тексту.
- Метод `split()` застосовують для розбиття рядка та підрахунку слів.
- Метод `strip()` допомагає очистити текст і видалити порожні рядки під час фільтрації.

### Загальне завдання:

Написати програму, яка зчитує дані з текстового файлу та записує їх у новий файл, виконуючи певні перетворення (наприклад, переведення всіх літер у нижній регістр).

**Індивідуальне завдання** - Записати список чисел у файл та відсортувати їх. (8 варіант)

### Текст програми:

```
# Лабораторна 5
# Тема: Робота з текстовими файлами: читання та запис
# Загальне завдання: Написати програму, яка зчитує дані з
текстового файлу та записує їх у новий файл,
# виконуючи певні перетворення (наприклад, переведення всіх
літер у нижній регістр).
# Варіант 8: Записати список чисел у файл та відсортувати їх.

import os

# Функція для відображення вмісту файлу в консолі (для
перевірки)
def print_file_content(filename, description):
    print(f"\nВміст файлу: {description} ({filename})")
```

```

if os.path.exists(filename):
    with open(filename, 'r', encoding='utf-8') as f:
        print(f.read())
else:
    print("Файл не знайдено.")
print("-" * 50)

# Функція для виконання загального завдання (зчитування та
# зміна реєстру).
def process_general_task(input_file, output_file):
    if not os.path.exists(input_file):
        print(f"Помилка: Файл '{input_file}' не знайдено.")
        return

    # Зчитуємо рядки з файлу
    with open(input_file, 'r', encoding='utf-8') as infile:
        lines = infile.readlines()

    # Переводимо всі літери у нижній реєстр
    processed_lines = [line.lower() for line in lines]

    # Записуємо результат у новий файл
    with open(output_file, 'w', encoding='utf-8') as outfile:
        outfile.writelines(processed_lines)

    print(f"Загальне завдання: Результат збережено у
    '{output_file}'")

# Функція для виконання індивідуального завдання
# (сортування чисел).
def process_individual_task(input_file, output_file):
    # Створюємо список чисел та записуємо його у файл за
    # умовою завдання
    raw_numbers = [45, 12, 89, 3, 22, 78, 1, 99, 34]
    with open(input_file, 'w', encoding='utf-8') as f:
        f.write(' '.join(map(str, raw_numbers)))

    if not os.path.exists(input_file):
        print(f"Помилка: Файл '{input_file}' не знайдено.")
        return

    # Зчитуємо дані з файлу
    with open(input_file, 'r', encoding='utf-8') as infile:
        data = infile.read()

```

```

        # Розбиваємо рядок на окремі елементи та перетворюємо
        # їх у числа
        numbers_list = [int(x) for x in data.split()]

        # Сортуємо список чисел
        numbers_list.sort()

        # Записуємо відсортовані числа у новий файл
        with open(output_file, 'w', encoding='utf-8') as outfile:
            outfile.write(' '.join(map(str, numbers_list)))

        print(f"Індивідуальне завдання: Числа відсортовано та
        збережено у '{output_file}'")

def main():
    # Підготовка даних для загального завдання
    input_gen = 'input_gen.txt'
    output_gen = 'output_gen.txt'

    with open(input_gen, 'w', encoding='utf-8') as f:
        f.write("ПЕРШИЙ Рядок ДЛЯ Тестування.\n")
        f.write("ДРУГИЙ рядок З ВЕЛИКИМИ і малими ЛІТЕРАМИ.\n")

    # Виклик функції загального завдання
    process_general_task(input_gen, output_gen)

    # Виклик функції індивідуального завдання
    input_ind = 'numbers_input.txt'
    output_ind = 'numbers_output.txt'
    process_individual_task(input_ind, output_ind)

    # Виведення результатів на екран для перевірки та звіту
    print_file_content(output_gen, "Загальне завдання (Нижній
    регістр)")
    print_file_content(output_ind, "Індивідуальне завдання
    (Відсортовані числа)")

# Перевірка прямого запуску файлу
if __name__ == "__main__":
    main()

```

## Скріншоти:

```
Загальне завдання: Результат збережено у 'output_gen.txt'
Індивідуальне завдання: Числа відсортовано та збережено у 'numbers_output.txt'

Вміст файлу: Загальне завдання (Нижній регістр) (output_gen.txt)
перший рядок для тестування.
другий рядок з великими і малими літерами.

-----

Вміст файлу: Індивідуальне завдання (Відсортовані числа) (numbers_output.txt)
1 3 12 22 34 45 78 89 99
-----

Process finished with exit code 0
```

## Висновок:

Під час виконання лабораторної роботи 5 я ознайомився з основами роботи з текстовими файлами в мові програмування Python. В ході роботи було опановано: методи відкриття файлів(навчився використовувати функцію `open()` з різними режимами доступу, такими як 'r' для читання та 'w' для запису), конструкцію `with`(закріпив навички використання контекстних менеджерів для

гарантованого та безпечного закриття файлів після завершення роботи з ними), обробку даних(реалізував алгоритми зчитування інформації методами `read()` та `readlines()`), а також навчився виконувати маніпуляції з текстом (переведення у нижній регістр) та числовими даними (сортування списку чисел), перевірку ресурсів(навчився використовувати модуль `os` для перевірки наявності файлів у системі, що дозволяє уникати критичних помилок під час виконання програми).